

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 2

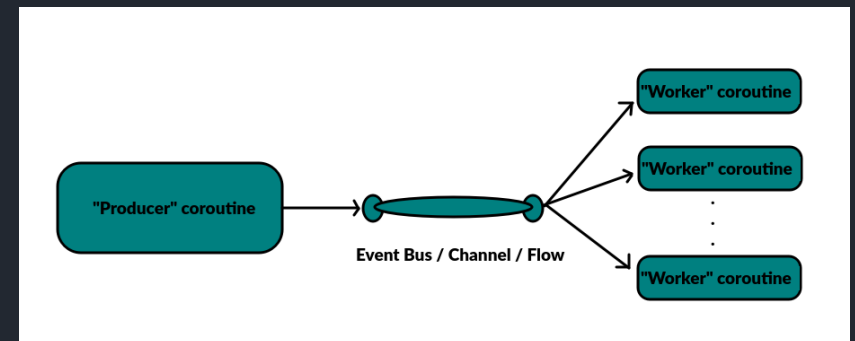
Advanced Kotlin, Coroutines, dan Flow

Pemrograman Asynchronous Modern untuk Aplikasi Multiplatform

Program Studi Teknik Informatika

Institut Teknologi Sumatera

Tahun Akademik Genap 2025/2026



OUTLINE PERTEMUAN

01

Review dan Advanced Kotlin

Null Safety, Extension Functions, Higher-Order Functions

02

Coroutines

Pemrograman Asynchronous yang Efisien

03

Kotlin Flow

Reactive Streams untuk Data Asynchronous

04

Hands-on Practice

3 Latihan Praktikum Terstruktur

05

Troubleshooting

Error Umum dan Solusinya

06

Tugas Praktikum

Penugasan Mingguan

CAPAIAN PEMBELAJARAN

CPMK0501

Mahasiswa mampu menerapkan konsep pemrograman untuk pengembangan perangkat lunak.

Indikator: Mampu mengimplementasikan coroutines dan flow dalam aplikasi KMP

CPMK0502

Mahasiswa mampu menjelaskan konsep pemrograman untuk pengembangan perangkat lunak.

Indikator: Mampu menjelaskan konsep asynchronous programming dan reactive streams

Setelah pertemuan ini, mahasiswa diharapkan dapat:

- ✓ Memahami dan menggunakan fitur advanced Kotlin (null safety, extension functions)
- ✓ Menjelaskan konsep dan implementasi Kotlin Coroutines
- ✓ Mengimplementasikan Kotlin Flow untuk data streaming
- ✓ Menerapkan pola asynchronous programming dalam aplikasi mobile

REVIEW MINGGU LALU

Apa yang sudah kita pelajari?

Kotlin Multiplatform

Framework untuk share code antar platform

Setup Environment

Android Studio, JDK, SDK

Struktur Proyek KMP

commonMain, androidMain, iosMain

Hello World App

Aplikasi pertama dengan KMP

Hari ini: Meningkatkan kemampuan Kotlin dengan Coroutines dan Flow!

ADVANCED KOTLIN FEATURES

Fitur-fitur Kotlin yang membuat kode lebih ekspresif dan aman

?

Null Safety

Menghindari NullPointerException dengan sistem tipe yang ketat

+

Extension Functions

Menambah fungsi baru ke class yang sudah ada tanpa inheritance

λ

Higher-Order Functions

Fungsi yang menerima atau mengembalikan fungsi lain

D

Data Classes

Class khusus untuk menyimpan data dengan boilerplate minimal

NULL SAFETY

💡 Analogi: Kotak Hadiah

Bayangkan kotak hadiah. Kotak biasa (String) PASTI berisi sesuatu. Kotak dengan tanda tanya (String?) MUNGKIN kosong. Kotlin memaksa kita untuk mengecek kotak "?" sebelum membukanya!

Kotlin

```
// Non-nullable - TIDAK boleh null
var nama: String = "Budi"
// nama = null // ERROR! Compile time error

// Nullable - BOLEH null (perhatikan tanda ?)
var alamat: String? = "Jakarta"
alamat = null // OK!

// Safe call operator (?.)
val panjang = alamat?.length // null jika alamat null

// Elvis operator (?:)
val panjangAman = alamat?.length ?: 0 // 0 jika null

// Not-null assertion (!!) - Gunakan dengan hati-hati!
val panjangPasti = alamat!!.length // Exception jika null
```

EXTENSION FUNCTIONS

💡 Analogi: Menambah Fitur ke Smartphone

Seperti menambah aplikasi ke smartphone tanpa mengubah sistem operasinya. Kita bisa menambah fungsi baru ke class String tanpa mengubah source code String!

Kotlin

```
// Menambah fungsi ke String
fun String.addExclamation(): String {
    return this + "!"
}

// Menambah fungsi ke Int
fun Int.isEven(): Boolean = this % 2 == 0

// Penggunaan
val greeting = "Hello"
println(greeting.addExclamation()) // Output: Hello!

val number = 4
println(number.isEven()) // Output: true

// Extension untuk nullable type
fun String?.orEmpty(): String = this ?: ""
val nama: String? = null
println(nama.orEmpty()) // Output: "" (string kosong)
```

HIGHER-ORDER FUNCTIONS

💡 Analogi: Mesin Kopi Otomatis

Seperti mesin kopi yang menerima "resep" (fungsi) untuk membuat berbagai jenis kopi. Mesin yang sama bisa membuat espresso atau cappuccino tergantung resep yang diberikan!

Kotlin

```
// Fungsi yang menerima fungsi lain sebagai parameter
fun operateOnNumbers(a: Int, b: Int, operation: (Int, Int) -> Int): Int {
    return operation(a, b)
}

// Penggunaan dengan lambda
val sum = operateOnNumbers(5, 3) { x, y -> x + y } // 8
val multiply = operateOnNumbers(5, 3) { x, y -> x * y } // 15

// Fungsi bawaan Kotlin yang higher-order
val numbers = listOf(1, 2, 3, 4, 5)

val doubled = numbers.map { it * 2 } // [2, 4, 6, 8, 10]
val evens = numbers.filter { it % 2 == 0 } // [2, 4]
val sum = numbers.reduce { acc, n -> acc + n } // 15

// forEach
numbers.forEach { println(it) }
```

LAMBDA EXPRESSIONS

Lambda adalah fungsi anonim yang bisa disimpan dalam variabel

Sintaks Lambda: `{ parameter1, parameter2 -> body }`

Jika hanya satu parameter, bisa menggunakan 'it' sebagai nama implisit

Kotlin

```
// Menyimpan lambda dalam variabel
val greet: (String) -> String = { name -> "Hello, $name!" }
println(greet("Kotlin")) // Output: Hello, Kotlin!

// Menggunakan 'it' untuk single parameter
val square: (Int) -> Int = { it * it }
println(square(5)) // Output: 25

// Lambda dengan multiple lines
val processData: (Int) -> Int = { number ->
    val doubled = number * 2
    val result = doubled + 10
    result // Nilai terakhir adalah return value
}

// Trailing lambda syntax
listOf(1, 2, 3).map { it * 2 } // Lambda sebagai argument terakhir
```

KOTLIN COROUTINES

Mengapa kita butuh Coroutines?

✗ Tanpa Coroutines

- UI Thread terblokir saat network call
- Aplikasi "Not Responding"
- User experience buruk
- Callback hell yang kompleks

✓ Dengan Coroutines

- Non-blocking async code
- UI tetap responsif
- Kode lebih mudah dibaca
- Structured concurrency

Definisi Coroutines

Coroutines adalah lightweight threads yang memungkinkan kita menulis kode asynchronous secara sequential (berurutan). Coroutines bisa di-suspend tanpa memblokir thread.

ANALOGI: RESTORAN DAN PELAYAN

Memahami Coroutines dengan Analogi Dunia Nyata

Thread = Pelayan

Pelayan tradisional menunggu di satu meja sampai pesanan selesai dimasak, baru pindah ke meja lain.

Thread yang terblokir = Pelayan yang mengganggu menunggu.

Coroutine = Pelayan Efisien

Pelayan efisien mencatat pesanan, lalu pindah ke meja lain selagi menunggu. Ketika masakan siap, ia kembali mengantarkan.

Coroutine di-suspend = Pelayan melayani meja lain.

Timeline Eksekusi:

Thread: [Meja 1 ] [Meja 2 ]

Coroutine: [Meja 1 ] [Meja 2 ] [Meja 1 ] [Meja 2 ] [Done!]

↑ suspend

↑ resume

SETUP COROUTINES DI KMP

Menambahkan dependency di build.gradle.kts (commonMain)

build.gradle.kts

```
kotlin {
    sourceSets {
        commonMain.dependencies {
            implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.8.0")
        }
    }
}
```

Konsep Kunci yang Akan Dipelajari:

CoroutineScope

Wadah untuk menjalankan coroutines

launch

Memulai coroutine baru (fire and forget)

async/await

Memulai coroutine yang mengembalikan hasil

suspend

Keyword untuk fungsi yang bisa di-pause

Dispatchers

Menentukan thread mana yang digunakan

COROUTINE BUILDERS

Cara memulai coroutine: launch vs async

launch { }

- Fire and forget
- Tidak mengembalikan hasil
- Mengembalikan Job
- Untuk tugas yang tidak butuh result

async { }

- Mengembalikan Deferred<T>
- Hasil diambil dengan .await()
- Untuk operasi paralel
- Ketika butuh hasil dari coroutine

Kotlin

```
import kotlinx.coroutines.*

fun main() = runBlocking { // CoroutineScope
    // launch - tidak mengembalikan hasil
    launch {
        delay(1000L)
        println("World!")
    }

    // async - mengembalikan hasil
    val result = async {
        delay(1000L)
        "Hello"
    }
    println(result.await()) // Menunggu dan mendapatkan hasil
}
```

SUSPEND FUNCTIONS

Apa itu Suspend Function?

Fungsi yang ditandai dengan keyword 'suspend' dapat di-pause (ditangguhkan) dan di-resume (dilanjutkan) tanpa memblokir thread. Suspend function hanya bisa dipanggil dari coroutine atau suspend function lain.

Kotlin

```
// Mendefinisikan suspend function
suspend fun fetchUserData(userId: String): User {
    delay(1000L) // Simulasi network call (non-blocking)
    return User(userId, "John Doe")
}

suspend fun fetchUserPosts(userId: String): List<Post> {
    delay(800L) // Simulasi network call
    return listOf(Post("Hello World"), Post("Kotlin is awesome!"))
}

// Penggunaan dalam coroutine
fun main() = runBlocking {
    val user = fetchUserData("123") // Sequential
    val posts = fetchUserPosts("123") // Menunggu user selesai

    println("User: ${user.name}")
    println("Posts: ${posts.size}")
}
```

COROUTINE DISPATCHERS

Dispatcher menentukan thread mana yang akan menjalankan coroutine

Dispatchers.Main

UI thread - untuk update tampilan

Update TextView, RecyclerView

Dispatchers.IO

I/O operations - network, database, file

API calls, baca/tulis file

Dispatchers.Default

CPU-intensive tasks

Sorting, parsing JSON besar

Tidak terikat thread tertentu

Testing (jarang dipakai)

```
// Contoh penggunaan dispatcher
viewModelScope.launch(Dispatchers.IO) {
    val data = repository.fetchData()
    withContext(Dispatchers.Main) {
        updateUI(data)
    }
}
```

```
// Jalankan di IO thread
// Network call
// Pindah ke Main thread
// Update UI
```

STRUCTURED CONCURRENCY

Coroutines mengikuti prinsip structured concurrency untuk menghindari memory leaks

Prinsip Structured Concurrency:

1. Setiap coroutine memiliki parent scope
2. Parent tidak selesai sampai semua children selesai
3. Jika parent dibatalkan, semua children juga dibatalkan
4. Jika child gagal, parent juga gagal (exception propagation)

Kotlin

```
fun main() = runBlocking {           // Parent scope
    launch {                          // Child 1
        delay(200L)
        println("Task 1 selesai")
    }
    launch {                          // Child 2
        delay(100L)
        println("Task 2 selesai")
    }
    println("Menunggu semua task...")
} // Parent menunggu kedua child selesai

// Output:
// Menunggu semua task...
// Task 2 selesai
// Task 1 selesai
```

Coroutines vs Flow

Suspend Function

Mengembalikan SATU nilai

```
suspend fun getUser(): User
```

↓ Returns: User

Flow

Mengembalikan BANYAK nilai secara berurutan

```
fun getUsers(): Flow<User>
```

↓ Emits: User1, User2, User3...

Definisi Flow

Flow adalah tipe data yang merepresentasikan stream (aliran) data yang dapat memancarkan (emit) multiple values secara sequential. Flow bersifat cold - hanya berjalan ketika ada collector.

Kapan menggunakan Flow?

Real-time updates dari database • Live data dari sensor • WebSocket messages • Periodic data refresh • Search suggestions

ANALOGI: ALIRAN SUNGAI

Memahami Flow dengan Analogi Air

Flow seperti Aliran Sungai

Producer (Sumber) = Mata air yang menghasilkan aliran

Flow = Sungai yang mengalirkan data terus-menerus

Operators = Bendungan/filter yang memodifikasi aliran

Collector = Waduk yang menampung dan menggunakan air

Visualisasi Flow:

```
[Producer]  →→→  [Operator]  →→→  [Operator]  →→→  [Collector]
```

<code>emit()</code>		<code>map()</code>		<code>filter()</code>		<code>collect()</code>
1,2,3	→	2,4,6	→	2,4,6	→	print()
		(kali 2)		(filter genap)		

MEMBUAT FLOW

Beberapa cara untuk membuat Flow:

Kotlin

```
import kotlinx.coroutines.flow.*

// 1. Menggunakan flow builder
fun numberFlow(): Flow<Int> = flow {
    for (i in 1..5) {
        delay(100)    // Simulasi operasi async
        emit(i)       // Memancarkan nilai
    }
}

// 2. Dari collection dengan .asFlow()
val listFlow = listOf(1, 2, 3).asFlow()

// 3. Menggunakan flowOf()
val simpleFlow = flowOf("A", "B", "C")

// 4. Flow dengan callbackFlow (untuk API callback-based)
fun locationUpdates(): Flow<Location> = callbackFlow {
    val listener = LocationListener { location ->
        trySend(location) // Kirim ke flow
    }
    // Setup listener...
    awaitClose { /* cleanup */ }
}
```

COLLECTING FLOW

PENTING: Flow bersifat COLD!

Flow tidak akan berjalan sampai ada collector. Setiap collector memulai flow dari awal.

Kotlin

```
fun main() = runBlocking {
    val numbersFlow = flow {
        println("Flow dimulai")
        for (i in 1..3) {
            delay(100)
            emit(i)
        }
    }

    // Collect adalah terminal operator
    println("Sebelum collect")

    numbersFlow.collect { value ->
        println("Received: $value")
    }

    println("Selesai")
}

// Output:
// Sebelum collect
// Flow dimulai
// Received: 1
// Received: 2
// Received: 3
// Selesai
```

FLOW OPERATORS

Operators untuk mentransformasi dan memfilter data dalam Flow

map

Transformasi setiap item

```
flow.map { it * 2 }
```

filter

Filter item berdasarkan kondisi

```
flow.filter { it > 5 }
```

take

Ambil n item pertama

```
flow.take(3)
```

onEach

Side effect untuk setiap item

```
flow.onEach { log(it) }
```

catch

Handle error dalam flow

```
flow.catch { emit(default) }
```

```
// Contoh kombinasi operators
(1..10).asFlow()
    .filter { it % 2 == 0 }    // 2, 4, 6, 8, 10
    .map { it * it }          // 4, 16, 36, 64, 100
    .take(3)                  // 4, 16, 36
    .collect { println(it) }
```

STATEFLOW DAN SHAREDFLOW

Hot Flows untuk state management di UI

StateFlow

- Menyimpan state terbaru
- Selalu punya nilai (initial value)
- Hanya emit jika nilai berubah
- Pengganti LiveData di KMP

SharedFlow

- Untuk one-time events
- Tidak menyimpan state
- Multiple subscribers
- Contoh: snackbar, navigation

Kotlin

```
class CounterViewModel {  
    // StateFlow dengan nilai awal  
    private val _count = MutableStateFlow(0)  
    val count: StateFlow<Int> = _count.asStateFlow()  
  
    // SharedFlow untuk events  
    private val _events = MutableSharedFlow<String>()  
    val events: SharedFlow<String> = _events.asSharedFlow()  
  
    fun increment() {  
        _count.value++ // Update state  
        viewModelScope.launch {  
            _events.emit("Counter updated") // Emit event  
        }  
    }  
}
```

HANDS-ON PRACTICE 1

Latihan: Coroutines Dasar

Tugas:

Buat program yang mengambil data dari 2 sumber secara paralel menggunakan `async/await`, kemudian gabungkan hasilnya.

Starter Code

```
suspend fun fetchUserProfile(userId: String): String {
    delay(1000) // Simulasi network delay
    return "User: John Doe"
}

suspend fun fetchUserPosts(userId: String): List<String> {
    delay(800) // Simulasi network delay
    return listOf("Post 1", "Post 2", "Post 3")
}

fun main() = runBlocking {
    // TODO: Jalankan kedua fungsi secara PARALEL dengan async
    // TODO: Gabungkan hasilnya
    // TODO: Ukur waktu eksekusi (harus < 2 detik jika paralel)

    val startTime = System.currentTimeMillis()
    // Kode kamu di sini...
    val endTime = System.currentTimeMillis()
    println("Waktu: ${endTime - startTime}ms")
}
```

SOLUSI HANDS-ON 1

Solusi

```
fun main() = runBlocking {
    val startTime = System.currentTimeMillis()

    // Jalankan paralel dengan async
    val profileDeferred = async { fetchUserProfile("123") }
    val postsDeferred = async { fetchUserPosts("123") }

    // Tunggu kedua hasil dengan await
    val profile = profileDeferred.await()
    val posts = postsDeferred.await()

    // Tampilkan hasil
    println(profile)
    println("Posts: $posts")

    val endTime = System.currentTimeMillis()
    println("Waktu eksekusi: ${endTime - startTime}ms")
    // Output: ~1000ms (bukan 1800ms karena paralel!)
}

// Penjelasan:
// - async memulai coroutine dan mengembalikan Deferred
// - await() menunggu hasil tanpa blocking
// - Kedua operasi berjalan bersamaan (paralel)
```

HANDS-ON PRACTICE 2

Latihan: Flow dengan Operators

Tugas:

Buat Flow yang mensimulasikan sensor suhu, filter suhu di atas 30°C, dan tampilkan warning dengan format yang bagus.

Starter Code

```
import kotlinx.coroutines.flow.*
import kotlin.random.Random

fun temperatureSensor(): Flow<Int> = flow {
    repeat(10) {
        delay(500)
        val temp = Random.nextInt(20, 40) // Random 20-39°C
        emit(temp)
    }
}

fun main() = runBlocking {
    // TODO: Gunakan operator flow untuk:
    // 1. Filter suhu > 30°C saja
    // 2. Transform menjadi string warning
    // 3. Tampilkan setiap warning

    temperatureSensor()
        // .filter { ... }
        // .map { ... }
        // .collect { ... }
}
```

SOLUSI HANDS-ON 2

Solusi

```
fun main() = runBlocking {
    temperatureSensor()
        .filter { temp ->
            temp > 30 // Hanya suhu di atas 30°C
        }
        .map { temp ->
            "⚠ WARNING: Suhu tinggi terdeteksi: ${temp}°C"
        }
        .onEach {
            println("Processing warning...") // Side effect
        }
        .collect { warning ->
            println(warning)
        }
}
```

```
// Contoh Output:
// Processing warning...
// ⚠ WARNING: Suhu tinggi terdeteksi: 35°C
// Processing warning...
// ⚠ WARNING: Suhu tinggi terdeteksi: 38°C
```

```
// Penjelasan:
// - filter: membuang suhu <= 30
// - map: transformasi Int ke String warning
// - onEach: side effect tanpa mengubah data
// - collect: terminal operator yang memicu flow
```

HANDS-ON PRACTICE 3

Latihan: StateFlow untuk Counter

Tugas:

Implementasikan counter sederhana menggunakan StateFlow. Counter harus bisa increment, decrement, dan reset.

Starter Code

```
import kotlinx.coroutines.flow.*

class CounterManager {
    // TODO: Buat MutableStateFlow dengan nilai awal 0
    // private val _count = ???

    // TODO: Expose sebagai StateFlow (read-only)
    // val count: StateFlow<Int> = ???

    fun increment() {
        // TODO: Tambah nilai count
    }

    fun decrement() {
        // TODO: Kurangi nilai count (minimum 0)
    }

    fun reset() {
        // TODO: Reset ke 0
    }
}
```

SOLUSI HANDS-ON 3

Solusi

```
class CounterManager {
    private val _count = MutableStateFlow(0)
    val count: StateFlow<Int> = _count.asStateFlow()

    fun increment() {
        _count.value++
    }

    fun decrement() {
        if (_count.value > 0) {
            _count.value--
        }
    }

    fun reset() {
        _count.value = 0
    }
}

// Penggunaan
fun main() = runBlocking {
    val counter = CounterManager()

    // Collect di background
    launch {
        counter.count.collect { println("Count: $it") }
    }

    counter.increment() // Count: 1
    counter.increment() // Count: 2
    counter.decrement() // Count: 1
    counter.reset()     // Count: 0
}
```

TROUBLESHOOTING

Error Umum dan Solusinya

✗ `Error: Suspend function can only be called from coroutine`

Penyebab: Memanggil suspend function dari fungsi biasa

Solusi: Panggil dari dalam coroutine scope (launch, runBlocking, viewModelScope)

✗ `Error: Flow emission from different coroutine is not allowed`

Penyebab: Emit dari coroutine yang berbeda

Solusi: Gunakan `channelFlow` atau `callbackFlow` untuk multi-coroutine emission

✗ `Error: Job was cancelled / CancellationException`

Penyebab: Parent scope dibatalkan sebelum child selesai

Solusi: Handle cancellation dengan try-finally atau gunakan `NonCancellable`

TROUBLESHOOTING (LANJUTAN)

✗ `Error: Flow invariant is violated (emission from different coroutine)`

Penyebab: Menggunakan launch di dalam flow builder

Solusi: Gunakan `channelFlow { }` untuk concurrent emission

💡 Best Practices

1. Selalu gunakan `viewModelScope` atau `lifecycleScope` di Android
2. Gunakan `Dispatchers.IO` untuk network dan database operations
3. Handle exceptions dengan `try-catch` atau `.catch` operator
4. Gunakan `StateFlow` untuk state, `SharedFlow` untuk events
5. Jangan lupa cancel coroutines yang tidak dibutuhkan
6. Test coroutines dengan `runTest` dan `TestDispatcher`
7. Hindari `GlobalScope` kecuali benar-benar diperlukan

RANGKUMAN

Advanced Kotlin

Null Safety (?.), Extension Functions, Higher-Order Functions, Lambda

Coroutines

launch, async/await, suspend functions, Dispatchers, CoroutineScope

Flow

Cold stream, emit/collect, operators (map, filter), StateFlow, SharedFlow

Key Takeaway

Coroutines dan Flow adalah fondasi untuk membuat aplikasi mobile yang responsif dan tidak memblokir UI thread.

TUGAS PRAKTIKUM

Bobot: 4% dari nilai akhir | Deadline: Pertemuan 3

Deskripsi Tugas

Buat aplikasi "News Feed Simulator" menggunakan Kotlin dengan fitur:

1. Flow yang mensimulasikan data berita baru setiap 2 detik
2. Filter berita berdasarkan kategori tertentu
3. Transform data menjadi format yang ditampilkan
4. StateFlow untuk menyimpan jumlah berita yang sudah dibaca
5. Coroutines untuk mengambil detail berita secara async

Pengumpulan: Upload ke GitHub dan submit link di LMS

Format: Proyek Kotlin dengan README yang menjelaskan cara menjalankan

RUBRIK PENILAIAN TUGAS

Komponen	Bobot	Kriteria
Implementasi Flow	25%	Flow builder, emit, collect benar
Penggunaan Operators	20%	filter, map, onEach tepat
StateFlow Implementation	20%	State management benar
Coroutines Usage	20%	async/await, dispatchers benar
Kode dan Dokumentasi	15%	Clean code, README lengkap

🌟 Bonus (+10%): Unit test untuk Flow dan Coroutines

Implementasi error handling dengan `.catch` dan exception handling

SUMBER PUSTAKA

Dokumentasi Resmi

Kotlin Coroutines Guide

kotlinlang.org/docs/coroutines-guide.html

Dokumentasi Resmi

Kotlin Flow Documentation

kotlinlang.org/docs/flow.html

Codelab

Advanced Coroutines with Kotlin Flow

developer.android.com/codelabs/advanced-kotlin-coroutines

Video

KotlinConf: Kotlin Flow in Practice

youtube.com/watch?v=fSB6_KE95bU

Artikel

StateFlow and SharedFlow

developer.android.com/kotlin/flow/stateflow-and-sharedflow

GitHub

Kotlin Coroutines Examples

github.com/Kotlin/kotlinx.coroutines/tree/master/kotlinx-coroutines-core

Terima Kasih

Pertemuan 2: Advanced Kotlin, Coroutines, dan Flow



Minggu Depan:

Compose Multiplatform Basics

Layouts, UI Components, dan Composable Functions

Ada pertanyaan? Silakan hubungi melalui email: muhammad.algifari@if.itera.ac.id